

Министерство образования и науки Российской Федерации  
Санкт-Петербургский политехнический университет Петра Великого

Институт компьютерных наук и кибербезопасности  
Высшая школа технологий искусственного интеллекта  
Направление: 02.03.01 «Математика и компьютерные науки»

Отчёт о выполнении лабораторных работ  
По дисциплине: «Методы проектирования баз данных»

Выполнил студент  
группы 5130201/40002

\_\_\_\_\_ Семенов И. А.

Проверил  
преподаватель

\_\_\_\_\_ Попов С. Г.

«\_\_\_\_\_» \_\_\_\_\_ 2026г.

# Содержание

|                                              |           |
|----------------------------------------------|-----------|
| <b>Введение</b>                              | <b>2</b>  |
| <b>1 Постановка задачи</b>                   | <b>3</b>  |
| <b>2 Основная часть</b>                      | <b>4</b>  |
| 2.1 Лабораторная работа № 1 . . . . .        | 4         |
| 2.1.1 Задача . . . . .                       | 4         |
| 2.1.2 Создание представления . . . . .       | 4         |
| 2.1.3 Проверка представления . . . . .       | 5         |
| 2.1.4 Запрос с условием . . . . .            | 7         |
| 2.1.5 Охват форматов по категориям . . . . . | 8         |
| 2.2 Лабораторная работа № 2 . . . . .        | 10        |
| 2.2.1 Задача . . . . .                       | 10        |
| 2.2.2 Схема базы данных . . . . .            | 10        |
| <b>3 Заключение</b>                          | <b>12</b> |

# Введение

# 1 Постановка задачи

## 2 Основная часть

### 2.1 Лабораторная работа № 1

#### 2.1.1 Задача

В ходе лабораторной работы необходимо:

1. создать представление, содержащее результат группировки данных с двумя агрегатными функциями `COUNT`;
2. составить простой запрос к созданному представлению с условием `WHERE`;
3. составить запрос, в котором представление используется совместно с операциями соединения `JOIN`.

#### 2.1.2 Создание представления

Необходимо создать представление, которое для каждого начертания хранит число поддерживаемых форматов и количество градаций насыщенности.

Одно начертание может храниться в таблице `Typeface` в нескольких форматах и градациях насыщенности. Чтобы не выполнять одинаковый подсчёт в каждом запросе, создаётся отдельное представление.

В нём для каждого начертания хранятся название, идентификатор гарнитуры, число форматов и число градаций насыщенности. Определение представления приведено в листинге 1.

В формулах используются обозначения:  $T$  — таблица `Typeface`,  $F$  — таблица `Format`,  $W$  — таблица `Weight`,  $FF$  — таблица `Font_family`,  $C$  — таблица `Category`,  $V$  — созданное представление. Символ  $\bowtie_k$  обозначает соединение по полю  $k$ ,  $\sigma$  — выборку,  $\pi$  — проекцию,  $\gamma$  — группировку с агрегированием.

Листинг 1: Создание представления

```
1 CREATE OR REPLACE VIEW typeface_format_weight_stats AS
2 SELECT
3     t.id_font_family,
4     t.name AS typeface_name,
5     COUNT(DISTINCT f.id_format) AS format_count,
6     COUNT(DISTINCT w.id_weight) AS weight_count
7 FROM typeface AS t
8 JOIN format AS f ON f.id_format = t.id_format
9 JOIN weight AS w ON w.id_weight = t.id_weight
10 GROUP BY
11     t.id_font_family,
12     t.name;
```

Агрегатная функция `COUNT(DISTINCT ...)` подсчитывает количество различных значений. Поэтому повторяющиеся форматы и градации насыщенности учитываются только один раз. Для получения фактических форматов и градаций насыщенности таблица `Typeface` соединяется с таблицами `Format` и `Weight`.

Создание представления описывается формулой

$$X = (T \bowtie_{id\_format} F) \bowtie_{id\_weight} W,$$

$$V = \gamma_{\substack{id\_font\_family, name; \\ count\_distinct(id\_format) \rightarrow format\_count, \\ count\_distinct(id\_weight) \rightarrow weight\_count}}(X).$$

### 2.1.3 Проверка представления

*Текст запроса:* Вывести первые десять начертаний из созданного представления вместе с числом поддерживаемых форматов и градаций насыщенности.

Запрос приведён в листинге 2.

Листинг 2: Проверка содержимого представления

```

1 SELECT
2     v.id_font_family,
3     v.typeface_name,
4     v.format_count,
5     v.weight_count
6 FROM typeface_format_weight_stats AS v
7 ORDER BY v.id_font_family, v.typeface_name
8 LIMIT 10;
```

Реляционная формула имеет вид

$$R_1 = \pi_{id\_font\_family, typeface\_name, \substack{format\_count, weight\_count}}(V).$$

План и результат выполнения показаны на рисунках 1 и 2.

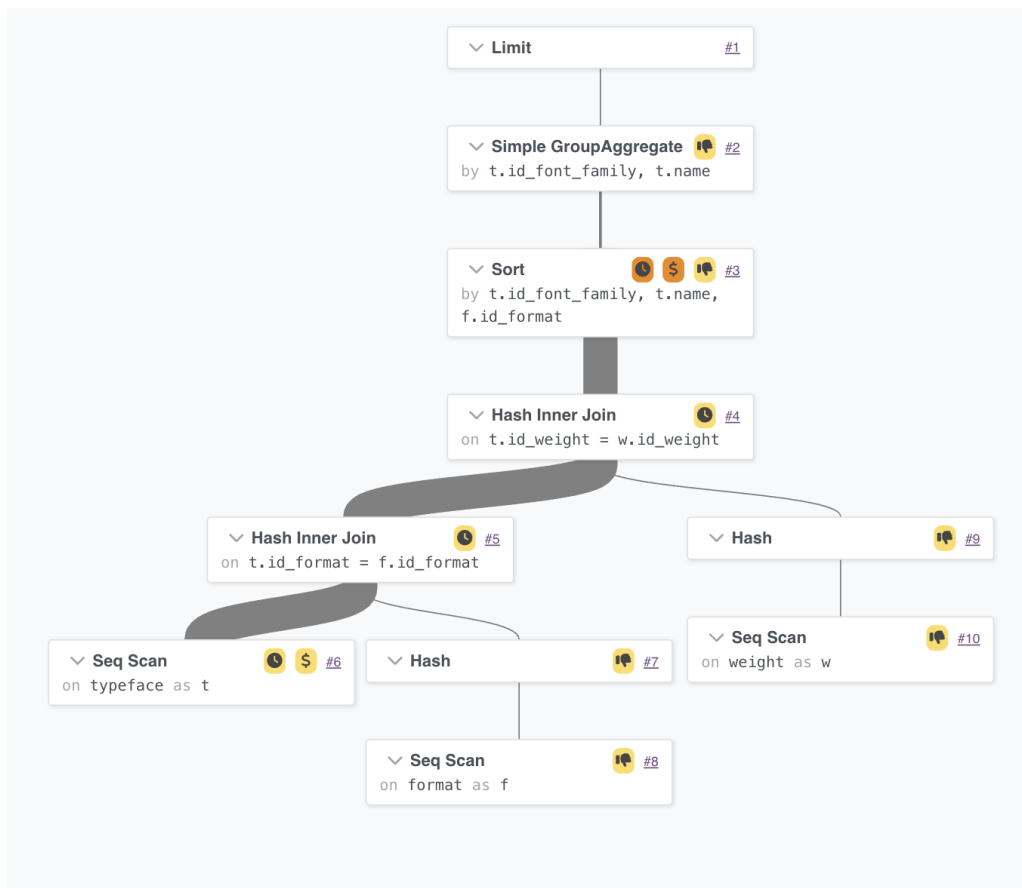


Рис. 1: План выполнения запроса 1

PostgreSQL читает таблицы `Typeface`, `Format` и `Weight`, затем соединяет их по полям `id_format` и `id_weight`. Полученные строки сортируются по полям группировки, после чего узел `GroupAggregate` рассчитывает количество различных форматов и градаций насыщенности каждого начертания.

cw-sem4 - zsh

✓ PostgreSQL: run current f...

/dev/fd/12:25: command not found: compdef

| id_font_family | typeface_name | format_count | weight_count |
|----------------|---------------|--------------|--------------|
| 1              | Typeface 1    | 1            | 5            |
| 2              | Typeface 2    | 1            | 2            |
| 3              | Typeface 3    | 1            | 9            |
| 4              | Typeface 4    | 1            | 7            |
| 5              | Typeface 5    | 1            | 5            |
| 6              | Typeface 6    | 1            | 8            |
| 7              | Typeface 7    | 1            | 7            |
| 8              | Typeface 8    | 1            | 9            |
| 9              | Typeface 9    | 1            | 5            |
| 10             | Typeface 10   | 1            | 2            |

(10 rows)

Рис. 2: Результат выполнения запроса 1

### 2.1.4 Запрос с условием

*Текст запроса:* Найти число начертаний, у которых количество поддерживаемых форматов больше  $A$ .

В запросе значение  $A$  принято равным 2. Сначала условие `format_count > 2` оставляет подходящие начертания, затем `COUNT` подсчитывает их. Запрос приведён в листинге 3.

Листинг 3: Подсчёт начертаний с числом форматов больше  $A$

```
1 SELECT
2     COUNT(v.id_font_family)
3 FROM typeface_format_weight_stats AS v
4 WHERE v.format_count > 2;
```

Запрос описывается формулой

$$R_2 = \gamma_{\text{count}(\text{id\_font\_family}) \rightarrow \text{typeface\_count}} (\sigma_{\text{format\_count} > A}(V)) .$$

План и результат выполнения показаны на рисунках 3 и 4.

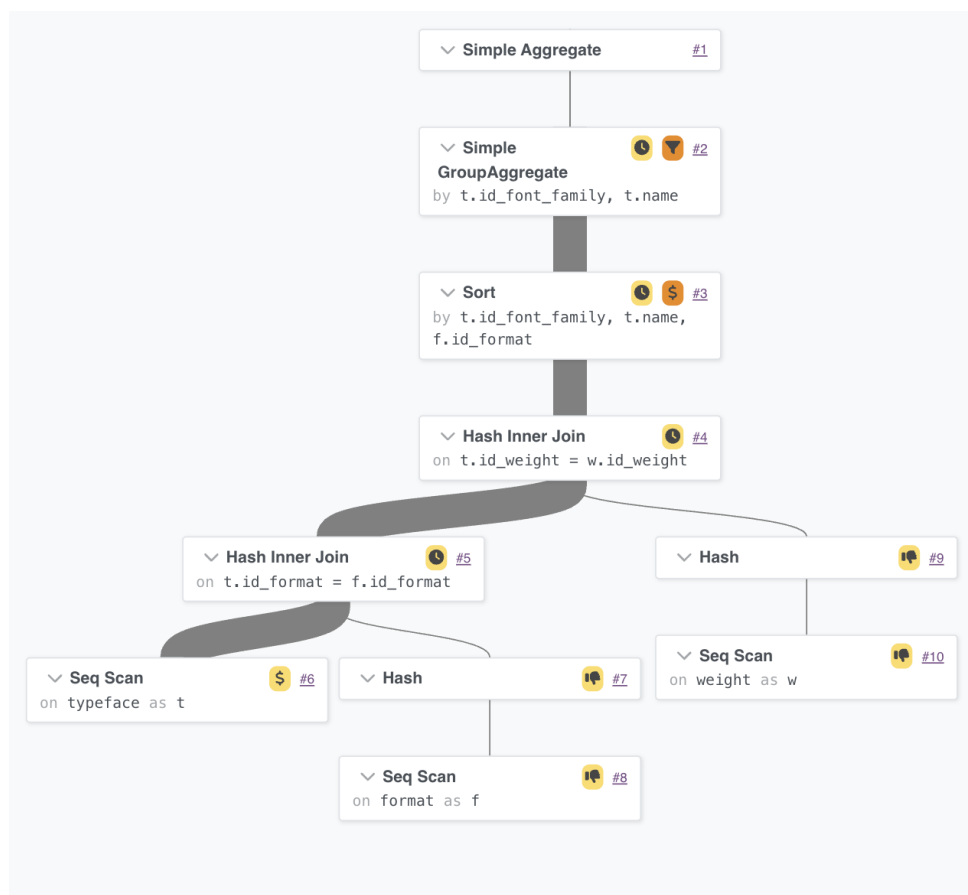


Рис. 3: План выполнения запроса 2

Сначала PostgreSQL соединяет таблицу начертаний с таблицами форматов и насыщенностей, после чего вычисляет строки представления с

помощью сортировки и `GroupAggregate`. Во время агрегирования отбрасываются начертания, у которых число форматов не больше  $A$ . Затем узел `Aggregate` подсчитывает оставшиеся строки.



```

> cw-sem4 - zsh      ✓ PostgreSQL: run current f...
/dev/fd/12:25: command not found: compdef
count
-----
      39
(1 row)

```

Рис. 4: Результат выполнения запроса 2

Условию соответствует 39 начертаний.

### 2.1.5 Охват форматов по категориям

*Текст запроса:* Найти пять категорий с наибольшим суммарным охватом поддерживаемых форматов.

Представление содержит идентификатор гарнитуры, поэтому его можно соединить с таблицами `Font_family` и `Category`. После соединения значения `format_count` суммируются отдельно для каждой категории. Запрос приведён в листинге 4.

Листинг 4: Первые пять категорий по охвату поддерживаемых форматов

```

1 SELECT
2     c.id_category,
3     c.name,
4     SUM(v.format_count) AS format_count
5 FROM typeface_format_weight_stats AS v
6 JOIN font_family AS ff
7     ON ff.id_font_family = v.id_font_family
8 JOIN category AS c
9     ON c.id_category = ff.id_category
10 GROUP BY c.id_category, c.name
11 ORDER BY format_count DESC, c.name
12 LIMIT 5;

```

Агрегирование по категориям и выбор первых пяти строк описываются формулами

$$\begin{aligned}
 X &= (V \bowtie_{id\_font\_family} FF) \bowtie_{id\_category} C, \\
 Q &= \gamma_{\substack{id\_category, name; \\ sum(format\_count) \rightarrow format\_count}}(X), \\
 R_3 &= top_5^{format\_count \downarrow, name \uparrow}(Q).
 \end{aligned}$$

Представление уже содержит число форматов каждого начертания, поэтому повторно считать их в запросе не требуется. План и результат выполнения показаны на рисунках 5 и 6.

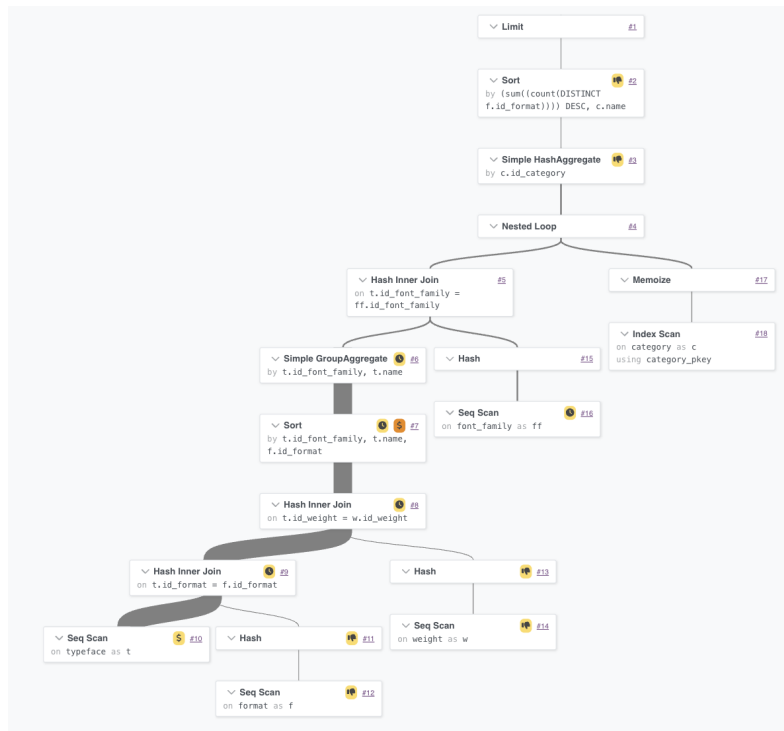


Рис. 5: План выполнения запроса 3

PostgreSQL вычисляет представление, соединяя начертания с форматами и насыщенностями. Полученные строки дополнительно соединяются с гарнитурами и категориями. Затем значения `format_count` суммируются по категориям. Завершающие сортировка и ограничение оставляют пять категорий с наибольшим охватом.

</

Рис. 6: Результат выполнения запроса 3

После группировки категории сортируются по убыванию охвата. Оператор `LIMIT` 5 оставляет первые пять строк. Наибольший охват в полученном результате имеет категория `Neo-grotesque`.

## 2.2 Лабораторная работа № 2

### 2.2.1 Задача

Необходимо реализовать таблицу `Typeface_format_weight_stats`, содержание которой повторяет результат представления, созданного в первой лабораторной работе. Таблица должна хранить для каждого начертания идентификатор гарнитуры, название начертания, число поддерживаемых форматов и количество градаций насыщенности.

Актуальность агрегированных данных необходимо обеспечить с помощью триггеров на таблицах `Font_family` и `Typeface`. При изменении исходных данных соответствующие строки таблицы статистики должны обновляться автоматически.

Требуется реализовать пять триггеров, покрывающих следующие изменения:

1. изменение идентификатора гарнитуры в таблице `Font_family`;
2. удаление гарнитуры из таблицы `Font_family`;
3. добавление начертания в таблицу `Typeface`;
4. изменение гарнитуры, названия, формата или насыщенности начертания в таблице `Typeface`;
5. удаление начертания из таблицы `Typeface`.

В рамках работы была подготовлена обновлённая схема базы данных: в неё добавлена таблица статистики, возле таблиц `Font_family` и `Typeface` указаны соответствующие триггеры, а их воздействие на агрегированные данные показано пунктирными стрелками.

### 2.2.2 Схема базы данных

В исходную схему добавлена таблица `Typeface_format_weight_stats`. Пунктирные стрелки показывают операции, при выполнении которых триггеры изменяют содержащиеся в ней агрегированные значения. Обновлённая схема представлена на рисунке 7.



### 3 Заключение